

SHOP BILLING SYSTEM

PROJECT REPORT & TECHNICAL DOCUMENTATION

Student Name: **Kiran Maurya**

Register No: **26boe10021**

Course: **Build Your Own Project (VITyarthi)**

Language: **Python 3**

1. EXECUTIVE SUMMARY & INTRODUCTION

The **Shop Billing System** is a modular, command-line interface (CLI) software application developed in Python. Designed primarily to serve small retail enterprises—such as local stationery shops, book stores, and convenience marts—it automates daily store transactions, inventory control, and financial reporting.

The system allows store owners to abandon paper-based ledger systems in favor of an automated inventory tracker, automated bill computation engine with built-in Goods and Services Tax (GST) and discount calculations, and persistent transaction logs.

2. PROBLEM STATEMENT

Small-scale vendors heavily rely on manual billing processes and paper notebooks. This introduces critical business operational flaws:

Calculation Errors

Manual calculation of item line totals, complex discount percentages, and GST tax additions leads to financial discrepancies.

Stock Overselling

Lack of real-time stock verification causes items to be sold past physical inventory levels, irritating customers.

No Inventory Visibility

Owners lack proactive alerts when high-demand stock is depleted, causing stock-outs and lost sales revenue.

Absence of Analytics

No systematic records exist to evaluate overall daily revenue, total sales count, or top-performing inventory items.

3. FUNCTIONAL REQUIREMENTS

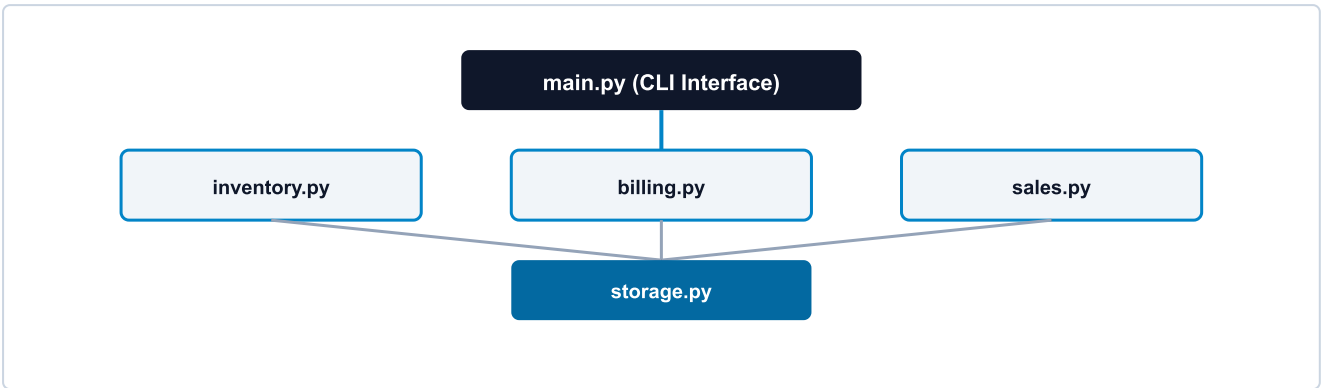
Module	Key Functions	Description
Module 1: Inventory inventory.py	Add, View, Remove, Reduce Stock	Manages catalog storage. Tracks unit prices and stock. Flags items with low stock (≤ 5 units).
Module 2: Billing billing.py	Build Bill, Apply Tax/Discount, Checkout	Builds customer cart, verifies stock availability, calculates subtotal, deducts discount, and adds 18% GST.
Module 3: Sales sales.py	Record Sale, View History, Export Summary	Logs historical transactions with dates, computes overall revenue, identifies best-sellers, and exports text reports.

4. NON-FUNCTIONAL REQUIREMENTS

- **Usability & Robustness:** Wrapped input prompts (`validators.py`) trap non-numeric data, empty strings, and out-of-range menu selections, ensuring zero runtime crashes during invalid user input.
- **Data Integrity & Persistence:** Centralized file storage layer (`storage.py`) manages JSON reading/writing with graceful fallback defaults if data files are missing or corrupted.
- **Maintainability & Modularity:** Decoupled design architecture with dedicated files handling configuration, storage, billing logic, catalog operations, and CLI interface routing.
- **Resource Efficiency:** Zero external package dependencies; written completely in standard Python 3, making it lightweight and runnable on any environment.

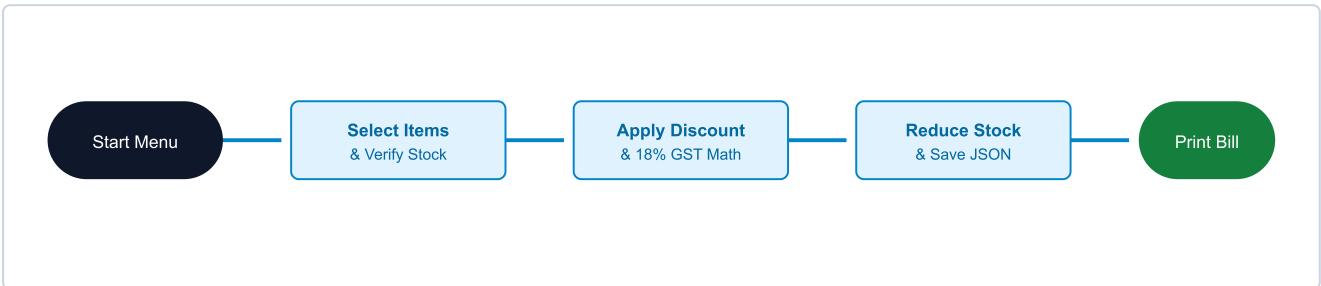
5. SYSTEM ARCHITECTURE

The system follows a layered architecture separating user input, core domain logic, and data storage.



6. DESIGN DIAGRAMS

6.1 Process Workflow Diagram



6.2 Database / JSON Schema Design

File Path	Data Structure	Sample Fields
<code>data/items.json</code>	Dictionary keyed by item name	<code>{"Pen": {"price": 10.0, "stock": 45}}</code>
<code>data/sales.json</code>	Array of transaction records	<code>[{"lines": [...], "subtotal": 130.0, "gst_amount": 21.06, "grand_total": 138.06, "date": "2026-09-29"}]</code>

7. KEY DESIGN DECISIONS & TECHNICAL RATIONALE

- **Pre-Tax Discount Sequence:** Discount percentage is subtracted from subtotal prior to GST application. Taxing customers on original price before discount is illegal and inaccurate.
 $\text{Taxable Amount} = \text{Subtotal} - \text{Discount Amount}$
 $\text{GST Amount} = \text{Taxable Amount} \times 18\%$
- **Isolated Input Validation:** Input functions (`ask_choice`, `ask_float`, `ask_int`, `ask_text`) in `validators.py` filter bad input before hitting logic.
- **Automated Stock Decrements:** Upon checkout completion, stock count is decremented automatically in memory and written to disk immediately to prevent stock drift.

8. EXECUTION OUTPUT & SAMPLE RESULT

```
===== SHOP BILLING SYSTEM
===== 1. Add item to catalog 2. View catalog 3. Remove
item 4. New bill (checkout) 5. View sales history 6. Sales summary 7. Export sales report 8.
Exit --- New Bill --- Available items: Notebook, Pen Item name (or type 'done' to finish):
Notebook Quantity (stock: 20): 2 Added 2 x Notebook to cart. Item name (or type 'done' to
finish): Pen Quantity (stock: 50): 3 Added 3 x Pen to cart. Item name (or type 'done' to
finish): done Discount percent (0 if none): 10 ===== BILL
===== Notebook 2 x 50.00 = 100.00 Pen 3 x 10.00 = 30.00 --
----- Subtotal : 130.00 Discount (10%) : -13.00 GST : 21.06
GRAND TOTAL : 138.06 =====
```

9. TESTING APPROACH & VERIFICATION

Automated unit tests were written in `tests/test_shop_billing.py` using Python's native `unittest` framework:

Test Suite	Test Case	Verified Behavior
InventoryTests	<code>test_reduce_stock_enough</code>	Verifies stock decreases correctly when quantity $\leq$ stock.
InventoryTests	<code>test_reduce_stock_not_enough</code>	Rejects transaction and preserves stock when requested qty > stock.
BillingTests	<code>test_discount_applied_before_gst</code>	Confirms 10% discount reduces taxable total before 18% GST math.

10. CHALLENGES FACED & SOLUTIONS

1. Tax Order Calculation

Challenge: Initial implementation computed GST on gross subtotal before discount, inflating final bills.

Solution: Restructured `build_bill()` in `billing.py` to calculate discount first, then compute 18% GST on net taxable sum.

2. Persistence Resilience

Challenge: Corrupted or empty JSON files caused `json.loads()` to throw unexpected runtime exceptions.

Solution: Wrapped loading logic inside `storage.py` with `JSONDecodeError` exception catching that safely defaults to empty catalog/sales objects.

11. KEY LEARNINGS

- **Modular Code Architecture:** Keeping catalog operations (`inventory.py`) separate from billing rules (`billing.py`) made testing each component straightforward.
- **Defensive Programming:** Sanitizing inputs at the entry layer prevents errors from polluting core business logic.
- **Practical Business Needs:** Gained hands-on experience structuring point-of-sale calculations, tax compliance rules, and low-stock warning thresholds.

12. FUTURE ENHANCEMENTS

1. **Flexible Discounting:** Support both percentage discounts and flat cash deductions (e.g., ₹50 flat discount).
2. **Invoice Generator:** Add auto-incrementing sequential invoice numbers (e.g., INV-2026-001) and PDF invoice exports.
3. **Return & Refund Module:** Allow shop owners to process item returns and automatically restore physical stock counts.

13. PROJECT REFERENCES

- VITyarthi - Build Your Own Project Instructions & Submission Guidelines Document.
- Python 3 Standard Library Documentation (`json`, `unittest`, `os`, `datetime` modules).
- Goods and Services Tax (GST) Business Calculation Rules in India.

Declaration: I hereby declare that this project report titled "**Shop Billing System**" is my original work created for the Build Your Own Project requirement under registration number **26boe10021**.

Student Name: Kiran Maurya | **Date:** 29th September 2026